security tests, compliance tests, and any other tests that might increase the confidence that the software can be released. If the mainline is always in a releasable state, releasing a new version of the software becomes a business decision instead of a technical one.

Continuous delivery encourages the automation of the whole process via a *deployment pipeline* (also called a *continuous delivery pipeline*), as described in the foundational book *Continuous Delivery* by Jez Humble and David Farley (Addison-Wesley Professional, 2010). A deployment pipeline goes from code commit to a releasable outcome, and it's the only way to production. Throughout the book, we'll build a deployment pipeline to keep the main branch of our applications always in a releasable state. In the end, we'll use it to deploy applications automatically to a Kubernetes cluster in production.

Sometimes continuous delivery is confused with *continuous deployment*. The former approach makes sure that after every change, the software is in a state in which it can be deployed to production. When that's actually done is a business decision. With continuous deployment we add one last step to the deployment pipeline to automatically deploy a new release in production after every change.

Continuous delivery is not about tools. It's a discipline that involves cultural and structural changes in your organization. Setting up an automated pipeline to test and deliver your applications doesn't mean that you're doing continuous delivery. Similarly, using a CI server to automate builds doesn't mean you're doing continuous integration.[12] That leads us to the next topic, which is also commonly mistaken as being about tools.

Continuous delivery vs. CI/CD